

Data Movement: MOV and LEA

■ MOV instruction: The workhorse.

- `mov rax, 42` (Immediate to register)
- `mov [var], rax` (Register to memory)

■ The Bracket Rule: In NASM, `[address]` means "the contents of memory at this address."

- `mov rax, my_var` → rax gets the address of the variable.
- `mov rax, [my_var]` → rax gets the value stored there.

■ LEA (Load Effective Address): Calculates an address without accessing memory.

- `lea rdi, [rbx + rcx * 4]` (Commonly used for fast pointer math).

Defining Data (Directives)

■ Initialized (.data):

- db (byte), dw (word), dd (dword), dq (qword).
- Example: msg db 'Hello', 10, 0

■ Uninitialized (.bss):

- resb, resw, resd, resq (reserve N units).
- Example: buffer resb 64

■ Equates:

- STDOUT equ 1 — Constant definition (pre-processor).

Arithmetic and Logic

- **Basic Ops: add, sub, imul, idiv.**
- **Logic: and, or, xor, not.**
- **XOR Trick: xor rax, rax is the standard way to set a register to zero (way shorter and faster than mov rax, 0).**
- **The Flags Register (RFLAGS):**
 - ZF (Zero Flag), SF (Sign Flag), CF (Carry Flag).
 - Updated automatically by arithmetic instructions.

Control Flow

- **Unconditional: jmp label**
- **Conditional: Based on CMP (Compare) or TEST.**
 - `cmp rax, rbx`
 - `je label` (Jump if equal / ZF=1)
 - `jne label` (Jump if not equal)
 - `jg / jl` (Signed Jump Greater/Less)
 - `ja / jb` (Unsigned Jump Above/Below)
- **Loops: While NASM has a loop instruction, manual dec and jnz are usually preferred for performance.**

Effective Addressing (Memory Access)

- **The Formula: $[\text{base} + \text{index} * \text{scale} + \text{displacement}]$**
- **Constraints:**
 - Base/Index: Any GPR.
 - Scale: Must be 1, 2, 4, or 8.
 - Displacement: Constant offset.
- **Example: `mov eax, [rbx + rcx*4 + 8]`**
 - Useful for array access (Base = array start, index = element index, scale = element size).

Jumping

■ jX Instructions

- Jump to different part of code depending on condition codes

jX	Condition	Description
jmp	1	Unconditional
j _e	ZF	Equal / Zero
j _{ne}	$\sim ZF$	Not Equal / Not Zero
j _s	SF	Negative
j _{ns}	$\sim SF$	Nonnegative
j _g	$\sim (SF \wedge OF) \ \& \ \sim ZF$	Greater (Signed)
j _{ge}	$\sim (SF \wedge OF)$	Greater or Equal (Signed)
j _l	$(SF \wedge OF)$	Less (Signed)
j _{le}	$(SF \wedge OF) \mid ZF$	Less or Equal (Signed)
j _a	$\sim CF \ \& \ \sim ZF$	Above (unsigned)
j _b	CF	Below (unsigned)

Conditional Branch Example

```
int absdiff(int x, int y)
{
    int result;
    if (x > y) {
        result = x-y;
    } else {
        result = y-x;
    }
    return result;
}
```

```
absdiff:
    pushl    %ebp
    movl     %esp, %ebp
    movl     8(%ebp), %edx
    movl     12(%ebp), %eax
    cmpl     %eax, %edx
    jle      .L6
    subl     %eax, %edx
    movl     %edx, %eax
    jmp      .L7
.L6:
    subl     %edx, %eax
.L7:
    popl     %ebp
    ret
```

Diagram illustrating the assembly code structure with labels and groupings:

- Setup**:
 - pushl %ebp
 - movl %esp, %ebp
- Body1**:
 - movl 8(%ebp), %edx
 - movl 12(%ebp), %eax
 - cmpl %eax, %edx
 - jle .L6
- Body2a**:
 - subl %eax, %edx
 - movl %edx, %eax
- Body2b**:
 - .L6: subl %edx, %eax
- Finish**:
 - .L7: popl %ebp
 - ret

Conditional Branch Example (Cont.)

```
int goto_ad(int x, int y)
{
    int result;
    if (x <= y) goto Else;
    result = x-y;
    goto Exit;
Else:
    result = y-x;
Exit:
    return result;
}
```

■ C allows “goto” as means of transferring control

- Closer to machine-level programming style

■ Generally considered bad coding style

absdiff:

pushl	%ebp	}	Setup
movl	%esp, %ebp		
movl	8(%ebp), %edx	}	Body1
movl	12(%ebp), %eax		
cmpl	%eax, %edx	}	Body2a
jle	.L6		
subl	%eax, %edx	}	Body2b
movl	%edx, %eax		
jmp	.L7	}	Finish
.L6:			
subl	%edx, %eax	}	Body2b
.L7:			
popl	%ebp	}	Finish
ret			

Conditional Branch Example (Cont.)

```
int goto_ad(int x, int y)
{
    int result;
    if (x <= y) goto Else;
    result = x-y;
    goto Exit;
Else:
    result = y-x;
Exit:
    return result;
}
```

```
absdiff:
    pushl    %ebp
    movl     %esp, %ebp
    movl     8(%ebp), %edx
    movl     12(%ebp), %eax
    cmpl     %eax, %edx
    jle      .L6
    subl     %eax, %edx
    movl     %edx, %eax
    jmp      .L7
.L6:
    subl     %edx, %eax
.L7:
    popl     %ebp
    ret
```

Setup

Body1

Body2a

Body2b

Finish

Conditional Branch Example (Cont.)

```
int goto_ad(int x, int y)
{
    int result;
    if (x <= y) goto Else;
    result = x-y;
    goto Exit;
Else:
    result = y-x;
Exit:
    return result;
}
```

```
absdiff:
    pushl    %ebp
    movl     %esp, %ebp
    movl     8(%ebp), %edx
    movl     12(%ebp), %eax
    cmpl     %eax, %edx
    jle      .L6
    subl     %eax, %edx
    movl     %edx, %eax
    jmp      .L7
.L6:
    subl     %edx, %eax
.L7:
    popl     %ebp
    ret
```

Setup

Body1

Body2a

Body2b

Finish

Conditional Branch Example (Cont.)

```
int goto_ad(int x, int y)
{
    int result;
    if (x <= y) goto Else;
    result = x-y;
    goto Exit;
Else:
    result = y-x;
Exit:
    return result;
}
```

```
absdiff:
    pushl    %ebp
    movl     %esp, %ebp
    movl     8(%ebp), %edx
    movl     12(%ebp), %eax
    cmpl     %eax, %edx
    jle      .L6
    subl     %eax, %edx
    movl     %edx, %eax
    jmp      .L7
.L6:
    subl     %edx, %eax
.L7:
    popl     %ebp
    ret
```

Setup

Body1

Body2a

Body2b

Finish

General Conditional Expression Translation

C Code

```
val = Test ? Then_Expr : Else_Expr;
```

```
val = x > y ? x - y : y - x;
```

Goto Version

```
nt = !Test;  
if (nt) goto Else;  
val = Then_Expr;  
goto Done;  
Else:  
    val = Else_Expr;  
Done:  
    . . .
```

- Test is expression returning integer
 - = 0 interpreted as false
 - ≠ 0 interpreted as true
- Create separate code regions for then & else expressions
- Execute appropriate one

Using Conditional Moves

■ Conditional Move Instructions

- Instruction supports:
if (Test) Dest $\leftarrow$ Src
- Supported in post-1995 x86 processors
- GCC does not always use them
 - Wants to preserve compatibility with ancient processors
 - Enabled for x86-64
 - Use switch `-march=686` for IA32

■ Why?

- Branches are very disruptive to instruction flow through pipelines
- Conditional move do not require control transfer

C Code

```
val = Test  
    ? Then_Expr  
    : Else_Expr;
```

Goto Version

```
tval = Then_Expr;  
result = Else_Expr;  
t = Test;  
if (t) result = tval;  
return result;
```

“Do-While” Loop Example

C Code

```
int pcount_do(unsigned x)
{
    int result = 0;
    do {
        result += x & 0x1;
        x >>= 1;
    } while (x);
    return result;
}
```

Goto Version

```
int pcount_do(unsigned x)
{
    int result = 0;
loop:
    result += x & 0x1;
    x >>= 1;
    if (x)
        goto loop;
    return result;
}
```

- Count number of 1's in argument x (“popcount”)
- Use conditional branch to either continue looping or to exit loop

“Do-While” Loop Compilation

Goto Version

```
int pcount_do(unsigned x) {  
    int result = 0;  
loop:  
    result += x & 0x1;  
    x >>= 1;  
    if (x)  
        goto loop;  
    return result;  
}
```

■ Registers:

%edx	x
%ecx	result

```
        movl    $0, %ecx        # result = 0  
.L2:    # loop:  
        movl    %edx, %eax  
        andl    $1, %eax        # t = x & 1  
        addl    %eax, %ecx      # result += t  
        shrl    %edx            # x >>= 1  
        jne     .L2            # If !0, goto loop
```

General “Do-While” Translation

C Code

```
do  
    Body  
while (Test) ;
```

Goto Version

```
loop:  
    Body  
    if (Test)  
        goto loop
```

■ **Body:** {
 Statement₁;
 Statement₂;
 ...
 Statement_n;
}

■ **Test returns integer**

- = 0 interpreted as false
- ≠ 0 interpreted as true

“While” Loop Example

C Code

```
int pcount_while(unsigned x) {  
    int result = 0;  
    while (x) {  
        result += x & 0x1;  
        x >>= 1;  
    }  
    return result;  
}
```

Goto Version

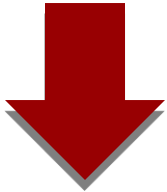
```
int pcount_do(unsigned x) {  
    int result = 0;  
    if (!x) goto done;  
loop:  
    result += x & 0x1;  
    x >>= 1;  
    if (x)  
        goto loop;  
done:  
    return result;  
}
```

■ Is this code equivalent to the do-while version?

General “While” Translation

While version

```
while (Test)  
    Body
```



Do-While Version

```
if (!Test)  
    goto done;  
do  
    Body  
    while (Test) ;  
done:
```



Goto Version

```
if (!Test)  
    goto done;  
loop:  
    Body  
    if (Test)  
        goto loop;  
done:
```

“For” Loop Example

C Code

```
#define WSIZE 8*sizeof(int)
int pcount_for(unsigned x) {
    int i;
    int result = 0;
    for (i = 0; i < WSIZE; i++) {
        unsigned mask = 1 << i;
        result += (x & mask) != 0;
    }
    return result;
}
```

- Is this code equivalent to other versions?

“For” Loop Form

General Form

```
for (Init; Test; Update)  
    Body
```

```
for (i = 0; i < WSIZE; i++) {  
    unsigned mask = 1 << i;  
    result += (x & mask) != 0;  
}
```

Init

```
i = 0
```

Test

```
i < WSIZE
```

Update

```
i++
```

Body

```
{  
    unsigned mask = 1 << i;  
    result += (x & mask) != 0;  
}
```

“For” Loop → While Loop

For Version

```
for (Init; Test; Update)  
    Body
```



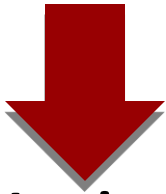
While Version

```
Init;  
while (Test) {  
    Body  
    Update;  
}
```

“For” Loop → ... → Goto

For Version

```
for (Init; Test; Update)  
    Body
```

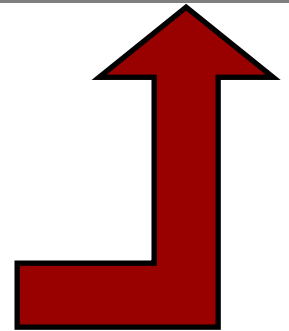


While Version

```
Init;  
while (Test) {  
    Body  
    Update;  
}
```



```
Init;  
if (!Test)  
    goto done;  
do  
    Body  
    Update  
while (Test);  
done:
```



```
Init;  
if (!Test)  
    goto done;  
loop:  
    Body  
    Update  
    if (Test)  
        goto loop;  
done:
```

“For” Loop Conversion Example

C Code

```
#define WSIZE 8*sizeof(int)
int pcount_for(unsigned x) {
    int i;
    int result = 0;
    for (i = 0; i < WSIZE; i++) {
        unsigned mask = 1 << i;
        result += (x & mask) != 0;
    }
    return result;
}
```

- Initial test can be optimized away

Goto Version

```
int pcount_for_gt(unsigned x) {
    int i;
    int result = 0;
    i = 0;
    if (!(i < WSIZE))
    goto done;
loop:
    {
        unsigned mask = 1 << i;
        result += (x & mask) != 0;
    }
    i++;
    if (i < WSIZE)
        goto loop;
done:
    return result;
}
```

Init

! Test

Body

Update

Test

```

long switch_eg
(long x, long y, long z)
{
    long w = 1;
    switch(x) {
    case 1:
        w = y*z;
        break;
    case 2:
        w = y/z;
        /* Fall Through */
    case 3:
        w += z;
        break;
    case 5:
    case 6:
        w -= z;
        break;
    default:
        w = 2;
    }
    return w;
}

```

Switch Statement Example

- Multiple case labels
 - Here: 5 & 6
- Fall through cases
 - Here: 2
- Missing cases
 - Here: 4

Jump Table Structure

Switch Form

```
switch(x) {  
  case val_0:  
    Block 0  
  case val_1:  
    Block 1  
    . . .  
  case val_n-1:  
    Block n-1  
}
```

Jump Table

jtab:	Targ0
	Targ1
	Targ2
	•
	•
	•
	Targn-1

Jump Targets

Targ0:

**Code Block
0**

Targ1:

**Code Block
1**

Targ2:

**Code Block
2**

•
•
•

Targn-1:

**Code Block
n-1**

Approximate Translation

```
target = JTab[x];  
goto *target;
```

Switch Statement Example (IA32)

```
long switch_eg(long x, long y, long z)
{
    long w = 1;
    switch(x) {
        . . .
    }
    return w;
}
```

What range of values takes default?

Setup:

```
switch_eg:
    pushl    %ebp                # Setup
    movl     %esp, %ebp         # Setup
    movl     8(%ebp), %eax       # %eax = x
    cmpl     $6, %eax           # Compare x:6
    ja       .L2                 # If unsigned > goto default
    jmp      *.L7(, %eax, 4)      # Goto *JTab[x]
```

Note that **w** not initialized here

Switch Statement Example (IA32)

```
long switch_eg(long x, long y, long z)
{
    long w = 1;
    switch(x) {
        . . .
    }
    return w;
}
```

Jump table

```
.section      .rodata
    .align 4
.L7:
    .long      .L2 # x = 0
    .long      .L3 # x = 1
    .long      .L4 # x = 2
    .long      .L5 # x = 3
    .long      .L2 # x = 4
    .long      .L6 # x = 5
    .long      .L6 # x = 6
```

Setup:

```
switch_eg:
    pushl      %ebp                # Setup
    movl      %esp, %ebp          # Setup
    movl      8(%ebp), %eax        # eax = x
    cmpl      $6, %eax            # Compare x:6
    ja        .L2                 # If unsigned > goto default
    Indirect  jump jump      *.L7(, %eax, 4) # Goto *JTab[x]
```

Assembly Setup Explanation

■ Table Structure

- Each target requires 4 bytes
- Base address at `.L7`

■ Jumping

- **Direct:** `jmp .L2`
- Jump target is denoted by label `.L2`
- **Indirect:** `jmp *.L7(, %eax, 4)`
- Start of jump table: `.L7`
- Must scale by factor of 4 (labels have 32-bits = 4 Bytes on IA32)
- Fetch target from effective Address `.L7 + eax*4`
 - Only for $0 \leq x \leq 6$

Jump table

```
.section .rodata
    .align 4
.L7:
    .long .L2 # x = 0
    .long .L3 # x = 1
    .long .L4 # x = 2
    .long .L5 # x = 3
    .long .L2 # x = 4
    .long .L6 # x = 5
    .long .L6 # x = 6
```

Jump Table

Jump table

```
.section .rodata
.align 4
.L7:
.long .L2 # x = 0
.long .L3 # x = 1
.long .L4 # x = 2
.long .L5 # x = 3
.long .L2 # x = 4
.long .L6 # x = 5
.long .L6 # x = 6
```

```
switch(x) {
case 1:      // .L3
    w = y*z;
    break;
case 2:      // .L4
    w = y/z;
    /* Fall Through */
case 3:      // .L5
    w += z;
    break;
case 5:
case 6:      // .L6
    w -= z;
    break;
default:    // .L2
    w = 2;
}
```

Handling Fall-Through

```
long w = 1;  
    . . .  
switch(x) {  
    . . .  
case 2:  
    w = y/z;  
    /* Fall Through */  
case 3:  
    w += z;  
    break;  
    . . .  
}
```

case 3:
 w = 1;
 goto merge;

case 2:
 w = y/z;
merge:
 w += z;

Code Blocks (Partial)

```
switch(x) {  
  case 1:      // .L3  
    w = y*z;  
    break;  
  . . .  
  case 3:      // .L5  
    w += z;  
    break;  
  . . .  
  default:    // .L2  
    w = 2;  
}
```

```
.L2:      # Default  
    movl $2, %eax # w = 2  
    jmp  .L8      # Goto done  
  
.L5:      # x == 3  
    movl $1, %eax # w = 1  
    jmp  .L9      # Goto merge  
  
.L3:      # x == 1  
    movl 16(%ebp), %eax # z  
    imull 12(%ebp), %eax # w = y*z  
    jmp  .L8      # Goto done
```

Summarizing

■ C Control

- if-then-else
- do-while
- while, for
- switch

■ Assembler Control

- Conditional jump
- Conditional move
- Indirect jump
- Compiler generates code sequence to implement more complex control

■ Standard Techniques

- Loops converted to do-while form
- Large switch statements use jump tables
- Sparse switch statements may use decision trees